

UNIVERSIDAD TÉCNICA FEDERICO SANTA MARÍA
DEPARTAMENTO DE INFORMÁTICA
CIUDAD - CHILE



“ASISTENTE DE ACCESIBILIDAD EN SITIOS WEB PARA PERSONAS CON DISCAPACIDAD VISUAL MEDIANTE EL USO DE INTELIGENCIA ARTIFICIAL”

IGNACIO BENJAMÍN RIQUELME VIDAL

MEMORIA PARA OPTAR AL TÍTULO DE
INGENIERO CIVIL EN INFORMÁTICA

Profesor Guía: Pedro Felipe Toledo Correa
Profesor Correferente: Jose Luis Martí Lara

Julio - 2026

DEDICATORIA

Considerando la importancia de este trabajo para los alumnos, este apartado es para que el autor entregue palabras personales para dedicar este documento. La extensión puede ser de máximo una hoja y se deben mantener este formato, tipo y tamaño de letra.

AGRADECIMIENTOS

Considerando la importancia de este trabajo para los alumnos, este apartado se podrá incluir en el caso de que el autor desee agradecer a las personas que facilitaron alguna ayuda relevante en su trabajo para la realización de este documento. La extensión puede ser de máximo una hoja y se deben mantener este formato, tipo y tamaño de letra.

RESUMEN

Resumen— El resumen y las palabras clave no deben superar la mitad de la página, donde debe precisarse brevemente: 1) lo que el autor ha hecho, 2) cómo lo hizo (sólo si es importante detallarlo), 3) los resultados principales, 4) la relevancia de los resultados. El resumen es una representación abreviada, pero comprensiva de la memoria y debe informar sobre el objetivo, la metodología y los resultados del trabajo realizado.

- Relator de pantalla: prototipo de herramienta que interpreta el contenido visual de sitios web y responde por voz a personas con discapacidad visual.
- Metodología de trabajo: encuesta -> prototipo -> benchmark -> validación con usuarios.
- Relevancia: >2.200 millones de personas con deficiencia visual; los lectores de pantalla no interpretan lo visual.

Palabras Clave— Cinco es el máximo de palabras clave para describir los temas tratados en la memoria, ponerlas separadas por punto y comas.

accesibilidad web; discapacidad visual; VLM; lectores de pantalla; benchmarking.

ABSTRACT

Abstract— Corresponde a la traducción al idioma inglés del Resumen anterior. Sujeto a la misma regla de extensión del Resumen.

Keywords— Corresponde a la traducción al idioma inglés de Palabras Clave anteriores.

GLOSARIO

Aquí se deben colocar las siglas mencionadas en el trabajo y su explicación, por orden alfabético. Por ejemplo:

DI: Departamento de Informática.

UTFSM: Universidad Técnica Federico Santa María.

ÍNDICE DE CONTENIDOS

RESUMEN	IV
ABSTRACT	IV
GLOSARIO	V
ÍNDICE DE FIGURAS	VIII
ÍNDICE DE TABLAS	VIII
INTRODUCCIÓN	1
CAPÍTULO 1: DEFINICIÓN DEL PROBLEMA	2
1.1 PROBLEMÁTICA SOCIAL	2
1.1.1 Antecedentes y encuesta propia	3
1.2 PROBLEMÁTICA TÉCNICA	4
1.3 OBJETIVOS Y ALCANCES	5
CAPÍTULO 2: MARCO CONCEPTUAL	7
2.1 LECTORES DE PANTALLA Y EL DOM	7
2.2 EXTENSIONES DE NAVEGADOR	8
2.3 RECONOCIMIENTO Y SÍNTESIS DE VOZ (STT Y TTS)	9
2.4 IA MULTIMODAL: MODELOS DE LENGUAJE VISUAL (VLM)	10
2.4.1 IA generativa	10
2.4.2 IA multimodal	10
2.4.3 Inferencia	11
2.5 EVALUACIÓN AUTOMATIZADA DE VLMs	11
2.5.1 Rúbrica cualitativa	11
2.5.2 LLM-as-judge	11
2.5.3 Judge evaluation	12
CAPÍTULO 3: PROPUESTA DE SOLUCIÓN	13
EJEMPLO DE COMO CITAR FIGURAS E ILUSTRACIONES	13
3.1 ARQUITECTURA GENERAL DEL RELATOR DE PANTALLA	14
3.2 EXTENSIÓN DE NAVEGADOR	14
3.2.1 textToText (TTT)	15
3.2.2 textToVoice (TTV)	16
3.2.3 voiceToVoice (VTV)	16
3.3 BACKEND DEL SISTEMA	17
3.4 INFRAESTRUCTURA Y SERVIDOR ALBERT	17
3.4.1 Decisión de infraestructura	17
3.5 BENCHMARK DE MODELOS VLM	18
3.5.1 Corpus y preguntas	18

3.5.2	Rúbrica de evaluación	19
3.5.3	Modelos evaluados	19
3.5.4	Pipeline y selección del juez (LLM-as-a-judge)	19
3.5.5	Resultado de la selección	20
CAPÍTULO 4: VALIDACIÓN DE LA SOLUCIÓN		21
4.1	EJEMPLO DE COMO CITAR TABLAS	21
4.2	ENFOQUE DE LA VALIDACIÓN	21
4.3	MÉTRICAS TÉCNICAS	22
4.3.1	Exactitud del reconocimiento de voz (STT)	22
4.3.2	Latencia del backend	22
4.3.3	Latencia de captura	22
4.3.4	Pertinencia de las respuestas	23
4.4	PRUEBAS CON USUARIOS: ENTREVISTAS	23
4.4.1	Diseño	23
4.4.2	Instrumento: puntaje de satisfacción	23
4.4.3	Participantes	24
4.4.4	Sesiones	24
4.4.5	Caso 1: primer usuario (P1)	24
4.4.6	Casos 2 a 4: P2 (dos sesiones) y P3	24
4.5	ANÁLISIS DE PATRONES	24
4.6	RETROALIMENTACIÓN DE LOS USUARIOS	25
4.7	CONCLUSIÓN DE LA VALIDACIÓN	25
CAPÍTULO 5: CONCLUSIONES		27
ANEXOS		28
5.1	ENCUESTA SOBRE TECNOLOGÍAS PARA PERSONAS CIEGAS	28
REFERENCIAS BIBLIOGRÁFICAS		29

ÍNDICE DE FIGURAS

1	Malla Curricular Ingeniería Civil Informática.	13
2	Arquitectura general del relator de pantalla.	15
3	Flujo de la información del relator de pantalla.	16

ÍNDICE DE TABLAS

1	Coloquios del Ciclo de Charlas Informática.	21
---	---	----

INTRODUCCIÓN

Debe proporcionar a un lector los antecedentes suficientes para poder contextualizar en general la situación tratada, a través de una descripción breve del área de trabajo y del tema particular abordado, siendo bueno especificar la naturaleza y alcance del problema; así como describir el tipo de propuesta de solución que se realiza, esbozar la metodología a ser empleada e introducir a la estructura del documento mismo de la memoria.

En el fondo, que el lector al leer la Introducción pueda tener una síntesis de cómo fue desarrollada la memoria, a diferencia del Resumen dónde se explicita más qué se hizo, no cómo se hizo.

- Área: cruce de accesibilidad web e IA (visión / VLM).
- Brecha: los lectores de pantalla no interpretan contenido visual; oportunidad de una capa intermediaria.
- Solución: el relator de pantalla como complemento, no reemplazo del lector de pantalla.
- Encuesta: punto de partida que motiva y valida el enfoque.
- Hoja de ruta: encuesta ->prototipo ->benchmark ->validación en más detalle.

CAPÍTULO 1

DEFINICIÓN DEL PROBLEMA

Se debe definir el problema, es importante no confundir definir el problema con describir la solución. Por ejemplo: “diseñar una arquitectura e implementar una plataforma ...” es una solución, no un problema.

Algunos elementos que podrían ir en este capítulo son (no es necesario que vayan todos):

- Breve descripción del contexto donde se realizará la memoria (organización, línea dentro de la Informática en la que se basa, etc.)
- ¿Qué y cómo se realiza actualmente la situación que mejorarás con tu memoria?
- ¿Qué actores o usuarios están involucrados?
- ¿Qué dificultades tienen esos actores actualmente? ¿cuántos son? (ideal si se pueden poner estadísticas para así saber si existe un mercado razonable para la solución que propondrás en tu memoria, en el fondo saber cuántas personas u organizaciones tienen el mismo problema que estás definiendo)
- ¿Qué podría pasar si en el corto o mediano plazo no se solucionan esas dificultades (¿es decir, si no se hiciera tu memoria, qué pasaría?; en el fondo justificar por qué conviene hacer tu memoria, ¿cuál es la motivación o interés de hacerla?).
- ¿Qué competencia existe actualmente? (a lo mejor ya existe una solución al problema, pero por qué no sirve, o por qué tu solución sería mejor, también se puede enfocar a si este problema existe en otras realidades y cómo ha sido solucionado allí).
- Precisar los objetivos y alcances de la memoria (o solución al problema).

En este capítulo, de ser necesario puede usar referencias bibliográficas (velar porque sean recientes), una cita de ejemplo [Schwab, 2002] y otras más [Georget *et al.*, 1994, Beaumont, 1990].

Recuerde poner notas al pie de página que sean explicativas ¹.

1.1. PROBLEMÁTICA SOCIAL

Las personas con discapacidad visual enfrentan barreras persistentes al interactuar con entornos digitales. La Organización Mundial de la Salud reporta que al menos

¹ Este es un ejemplo de una nota al pie de página. Puede indicar alguna URL, definiciones, aclarar alguna información pertinente del texto, citar algunas referencias, etc..

2.200 millones de personas en el mundo tienen algún grado de **deficiencia** visual [World Health Organization, 2026], pero esa cifra agrupa mayoritariamente casos leves y corregibles con lentes. Al filtrar por los niveles de discapacidad severa y ceguera total, las estimaciones del Global Burden of Disease para el año 2020 son de 43,3 millones de personas ciegas y 295 millones con discapacidad visual moderada a severa, es decir, un orden de 338 millones de personas a nivel mundial [Bourne *et al.*, 2021].

Sin embargo, la magnitud de esta cifra no es el enfoque, sino la **pertinencia** ética de integrar a un grupo de usuarios históricamente segregado del entorno tecnológico general. El problema de fondo no es que existan herramientas pensadas específicamente para personas ciegas, sino el acceso desigual a ellas: alternativas de uso extendido a nivel profesional, como JAWS, requieren de una licencia **paga** [Freedom Scientific, 2024]. A esta barrera económica se suma una barrera de aprendizaje: mientras que una persona sin discapacidad visual puede aprender a usar un computador de forma prácticamente autodidacta, dominar un lector de pantalla implica memorizar **cientos** de combinaciones de teclado y, con frecuencia, tomar cursos de capacitación formal —dictados por organizaciones especializadas o por el propio fabricante— antes de poder utilizarlo con fluidez. Evitar segregar o estigmatizar a un grupo de usuarios mediante soluciones separadas ha sido, de hecho, uno de los principios formales del diseño universal —el principio de uso equitativo— en los últimos 20 años [Mace y The Center for Universal Design, 1997].

La consecuencia de no resolver estas barreras es la exclusión en las dimensiones social, laboral y educativa: la imposibilidad de acceder de forma autónoma a contenido en línea limita la participación social cotidiana, el desempeño y las oportunidades laborales, y el acceso a instancias educativas. A modo de ejemplo, hoy no es posible enterarse por cuenta propia de qué muestra una fotografía que comparte un familiar o un amigo, ni completar sin ayuda una compra o un trámite en línea cuando de por medio se exige una verificación visual [WebAIM, 2024]. Tampoco es posible darse cuenta a tiempo de un aviso que aparece de forma inesperada en la pantalla, lo que puede significar perder una oportunidad o un plazo importante. En el trabajo, muchas veces es necesario pedirle a un compañero que describa una cifra o una imagen antes de poder tomar una decisión. Algo similar ocurre en lo educativo, donde acceder a un material de estudio compartido como imagen deja a la persona a la espera de que alguien más se lo describa antes de poder seguir aprendiendo al mismo ritmo que el resto.

1.1.1. Antecedentes y encuesta propia

El punto de partida se basó en que esta problemática social afecta de manera particular a las personas ciegas al navegar la web. Sin embargo, al revisar el estado del arte no se encontraron estudios específicos que caracterizaran con suficiente detalle esa necesidad en este grupo, sobre todo porque ya existen soluciones tecnológicas que resuelven parte de esto,

como por ejemplo los lectores de pantalla². Ante la ausencia de antecedentes que caracterizaran esta necesidad con suficiente detalle, se diseñó y aplicó una encuesta propia con el fin de **validarla** empíricamente antes de proponer una solución.

Se elaboró y distribuyó el instrumento “Encuesta sobre tecnologías para personas ciegas” mediante Google Forms, enviado por correo electrónico a **miembros** de **FUNDALURP** (Fundación de Lucha contra la Retinitis Pigmentosa), organización dedicada a la rehabilitación de personas en situación de discapacidad visual. La encuesta se aplicó entre febrero y marzo de 2026 y se obtuvieron 9 **encuestados**. El detalle completo del instrumento —su estructura, tipos de pregunta y cómo se relacionan con la problemática— se presenta en el Anexo 5.1.

El perfil de los encuestados muestra como condición dominante la retinitis pigmentaria, con niveles de discapacidad que van desde severo hasta ceguera total, y un manejo computacional autoreportado mayoritariamente entre básico e intermedio. Respecto del lector de pantalla utilizado, **casi ningún encuestado reportó usar JAWS**, optando en su lugar por alternativas sin costo como NVDA. **pendiente: agregar conteo o porcentaje exacto de encuestados por lector de pantalla utilizado.**

Respecto de las dificultades reportadas al navegar la web con lectores de pantalla, las más frecuentes, **en orden**, fueron: elementos estrictamente visuales que no pueden interpretarse (imágenes, gráficos); pop-ups o notificaciones que no se leen por falta de etiquetado `aria-label`; campos de escritura sin etiqueta; la obligación de usar el ratón para ciertas interacciones; y los captchas visuales. **pendiente: agregar conteo o porcentaje exacto de encuestados que reportó cada dificultad.**

En cuanto a las funcionalidades identificadas **como las más valoradas**, se destacan tres, en orden de **frecuencia**: describir e interpretar imágenes; la navegación por voz hacia un objetivo concreto; y la lectura de notificaciones o etiquetas dinámicas. **pendiente: agregar el conteo o porcentaje exacto de encuestados que valoró funcionalidad.**

1.2. PROBLEMÁTICA TÉCNICA

Actualmente, las personas con discapacidad visual navegan la web apoyándose en lectores de pantalla [Abou-Zahra y World Wide Web Consortium (W3C) Web Accessibility Initiative, 2024], programas como JAWS, NVDA o VoiceOver que traducen el contenido de una página a voz sintetizada o a un dispositivo braille —**una línea física** con celdas que forman y deshacen caracteres en relieve mediante pines accionados electromecánicamente, y que se conecta al computador para ir mostrando en tiempo real, al tacto, el contenido que el lector de pantalla va comunicando—. **El DOM (Document Object Model)** es la estructura de nodos que el navegador construye a partir del código HTML de la página en su parte estática inicial, pero que también tiene una dimensión programática: JavaScript puede modificar ese

²Programas que traducen el contenido de una página web a voz sintetizada o a un dispositivo braille; se profundiza en su funcionamiento en el Capítulo 2.

DOM en tiempo de ejecución, agregando, quitando o alterando nodos después de la carga inicial. A partir del DOM así construido, el navegador deriva un árbol de accesibilidad —una representación paralela que expone únicamente los roles, nombres accesibles, estados y relaciones relevantes para una tecnología asistiva— y lo pone a disposición a través de la API de accesibilidad del sistema operativo [MDN Web Docs, 2024a]. Es esa API la que el lector de pantalla consulta para construir lo que finalmente comunica al usuario. También existe la posibilidad de que el usuario navegue por una página utilizando únicamente el teclado, lo cual es posible gracias al atributo `tabindex` y al orden del DOM. El recorrido secuencial mediante la tecla Tab lo controla el propio navegador para la navegación por teclado, en el orden del DOM o según el atributo `tabindex`, y no es exclusivo de los lectores de pantalla ni de los usuarios con discapacidad visual.

Buena parte de la información que un lector de pantalla puede comunicar depende de las etiquetas ARIA (Accessible Rich Internet Applications): un conjunto de atributos que se agregan a los elementos del DOM para declarar roles, estados y relaciones que el HTML sin marcadores adicionales no permite describir por sí solo [MDN Web Docs, 2024b]. Por ejemplo, una notificación emergente puede marcarse como una región viva (`aria-live`) para que el lector la anuncie apenas aparece. Cuando un sitio no declara estas etiquetas, el lector de pantalla no tiene forma de identificar ese contenido dinámico, aunque técnicamente sí forme parte del DOM.

El límite de este enfoque está, entonces, en que depende por completo de que la página declare una semántica correcta y suficiente para cada elemento —ya sea mediante etiquetas HTML semánticas (por ejemplo `<article>` o `<dialog>`) o mediante ARIA. Los lectores de pantalla no interpretan información estrictamente visual: no describen imágenes, gráficos ni captchas, y solo leen de forma confiable las notificaciones o pop-ups que fueron etiquetados correctamente por quien construyó el sitio. Tampoco comprenden la estructura espacial de la interfaz (qué elemento está dónde y con qué propósito) más allá del orden en que fueron declarados. El usuario recibe así una representación incompleta de la página, que depende de decisiones de accesibilidad ajenas a su control.

Respecto de la competencia y soluciones existentes, las alternativas actuales resultan insuficientes frente a estas brechas de accesibilidad [Campbell *et al.*, 2024]. Los lectores de pantalla, por diseño, no cubren la interpretación visual; y si bien existen soluciones basadas en IA para solicitar descripciones de imagen, estas no están integradas dentro del flujo de navegación con lector de pantalla ni fueron pensadas específicamente para personas ciegas, por lo que no resuelven el problema de forma completa.

1.3. OBJETIVOS Y ALCANCES

Objetivo general. Desarrollar una herramienta denominada relator de pantalla, que permite acceder a información descriptiva de un sitio web mediante interacciones de pregunta y respuesta en lenguaje natural, y que se integre y coexista con el lector de pantalla del usuario.

Objetivos específicos.

- (a) Implementar un prototipo funcional del relator de pantalla capaz de interpretar información visual y entregar descripciones auditivas.
- (b) Ajustar el comportamiento del **modelo**, mediante el afinamiento de sus instrucciones, para adaptarse de mejor forma a las necesidades específicas del usuario.
- (c) Empaquetar e **implementar** la solución para una disponibilización básica entre usuarios.
- (d) Validar la solución mediante pruebas con usuarios reales.

Alcances. Se espera que las personas ciegas puedan interactuar de forma más directa con el computador para acceder a información que el lector de pantalla actualmente no les puede proveer de forma nativa, facilitando el acceso a información visual de la pantalla mediante interacciones fáciles **para ellas**.

CAPÍTULO 2

MARCO CONCEPTUAL

Se debe describir la base conceptual o fundamentos en los que se basa tu memoria, es decir, todos los conceptos técnicos, metodologías, herramientas, etc. que están involucradas en la solución propuesta. En el fondo esta parte permite precisar y delimitar el problema, estableciendo definiciones para unificar conceptos y lenguaje y fijar relaciones con otros trabajos o soluciones encontradas por otros al mismo problema evitando así plagios o repetir errores ya conocidos o abordados por otros.

En esta parte es importante relacionar estos conceptos con la memoria y es fundamental utilizar referencias bibliográficas (o de la web) recientes, por ejemplo [Gettelfinger y Cussler, 2004].

2.1. LECTORES DE PANTALLA Y EL DOM

Un lector de pantalla es un software de tecnología asistiva que convierte el contenido de una interfaz digital en voz sintetizada o en salida braille, permitiendo al usuario acceder a esa información sin necesidad de verla [Abou-Zahra y World Wide Web Consortium (W3C) Web Accessibility Initiative, 2024].

La salida braille se entrega mediante una línea braille refrescable: un dispositivo de hardware, conectado al computador, con celdas que forman y deshacen caracteres en relieve mediante pines accionados electromecánicamente, y que va mostrando al tacto y en tiempo real el contenido que el lector de pantalla comunica.

Los lectores de pantalla no son una categoría homogénea ni universal: existen embebidos en el propio sistema operativo, como Windows Narrator o VoiceOver en macOS/iOS, y otros que se instalan por separado, como JAWS o NVDA; dentro de estos últimos hay opciones de código abierto y sin costo (NVDA) y opciones pagas orientadas al uso profesional (JAWS); y hay tanto lectores de escritorio como lectores para dispositivos móviles, como TalkBack en Android. Qué lector de pantalla resulta más adecuado depende, entonces, del sistema operativo, el dispositivo y el presupuesto disponible, y no existe uno que cubra todos los contextos de uso por igual.

[tabla: clasificación de lectores de pantalla más usados (JAWS, NVDA, Narrator, VoiceOver, TalkBack) según si son embebidos del sistema operativo o instalados, de código abierto o pagos, y de escritorio o móviles.]

El DOM (Document Object Model) es la representación estructurada del documento web: una jerarquía de nodos (elementos, atributos, texto) que el navegador construye a partir del código HTML de la página en su parte estática inicial, y sobre la que se puede consultar y

operar mediante programación [WHATWG, 2026]. Esa segunda dimensión, programática, es la que aporta JavaScript: al ejecutarse, puede agregar, quitar o modificar nodos del DOM ya construido, de modo que lo que el navegador termina exponiendo no depende solo del HTML servido inicialmente, sino también de los cambios que JavaScript introduce después. El navegador deriva del DOM resultante un árbol de accesibilidad, una representación paralela y más acotada que expone únicamente los roles, nombres, estados y relaciones relevantes para una tecnología asistiva, a través de la API de accesibilidad del sistema operativo (por ejemplo, UI Automation en Windows o AXAPI en macOS). Es esa API la que el lector de pantalla consulta para construir lo que finalmente comunica al usuario.

La capacidad del navegador de construir un árbol de accesibilidad completo depende de que la página declare correctamente su semántica, algo que el HTML sin marcadores adicionales no siempre garantiza. El W3C define para ello dos estándares complementarios: las Pautas de Accesibilidad para el Contenido Web (WCAG), que establecen los criterios que debe cumplir un sitio para considerarse accesible [Campbell *et al.*, 2024], y WAI-ARIA, un conjunto de atributos (roles, estados y propiedades `aria-*`) que permite declarar la semántica de elementos que no la expresan por sí solos —por ejemplo, marcar una notificación emergente como una región viva (`aria-live`) para que se anuncie apenas aparece— [Nurthen *et al.*, 2023]. El atributo `alt`, por su parte, es el mecanismo nativo para asociar un texto alternativo a una imagen —y existe desde antes que los lectores de pantalla, no exclusivamente para ellos—; sin él, una imagen no tiene ninguna representación textual que el lector de pantalla pueda comunicar [World Wide Web Consortium (W3C), Web Accessibility Initiative, 2014].

El lector de pantalla permite al usuario recorrer **secuencialmente** el contenido de la página mediante la tecla Tab, uno tras otro: no solo los elementos interactivos como campos de texto o enlaces, sino también contenido no interactuable como párrafos de texto. Ese orden de recorrido no es estrictamente el orden en que los elementos fueron declarados en el HTML: el atributo `tabindex` permite alterarlo, adelantando o postergando elementos respecto de su posición original en el documento. Además de ese recorrido, el lector de pantalla ofrece ayudas de navegación que no existen sin él: una de las más usadas es el listado de enlaces, que presenta todos los enlaces de la página en una lista y permite saltar directamente al que se necesite.

La eficacia de este recorrido secuencial depende de la familiaridad previa del usuario con la estructura de la página: sobre un sitio ya explorado, permite ubicar rápidamente los elementos buscados; frente a un sitio nuevo, requiere recorrerlo y estudiarlo varias veces antes de poder anticipar su estructura.

2.2. EXTENSIONES DE NAVEGADOR

Una extensión es un módulo de software que un usuario instala sobre su navegador para modificar o ampliar su comportamiento —por ejemplo, para bloquear anuncios, gestionar contraseñas o capturar información de una página—, sin ser una aplicación independiente

ni una función nativa del propio navegador [MDN Web Docs, 2024c].

Google Chrome fue el primer navegador en incorporar un sistema de extensiones basado enteramente en HTML, CSS y JavaScript, habilitado en su canal de desarrollo en 2009 y lanzado de forma pública junto a Chrome 4.0 en enero de 2010 [Chromium Blog, 2010]. A partir de esa base, Mozilla adoptó en 2015 una API compatible para su navegador Mozilla Firefox, bautizada WebExtensions, que buscaba convertirse en un estándar común entre navegadores; en 2021 se llegó a formar un grupo comunitario del W3C específicamente para intentar formalizar esa base común entre distintos fabricantes [World Wide Web Consortium (W3C), 2021], pero ese intento de estandarización no llegó a concretarse. Pese a ello, la popularidad de Chrome llevó a que el resto de los navegadores basados en su motor Chromium (Edge, Opera, Brave, entre otros) convergieran hacia una API de extensiones muy similar [MDN Web Docs, 2024c].

Hoy existen, en términos generales, dos grandes familias de navegador incompatibles entre sí: los basados en Chromium, que agrupan a la gran mayoría, y Mozilla Firefox, basado en el motor Gecko y heredero del antiguo navegador Netscape. Aparte de estas dos familias existen otros navegadores, como los propietarios de ciertos fabricantes de dispositivos móviles. Un caso aparte es Safari que utiliza su propio motor, WebKit. Independiente de que un navegador soporte extensiones, estas solo son compatibles con el navegador para el que fueron diseñadas: el código y el empaquetado de una extensión para Chrome no son directamente instalables en Firefox ni viceversa, aún cuando ambas compartan la mayoría de las llamadas de su API.

A nivel de capacidades, una extensión se ejecuta bajo un modelo de permisos explícitos declarados en un archivo de manifiesto: solo puede acceder a las páginas, pestañas o funciones del navegador para las que fue autorizada, y se organiza mediante scripts de contenido — que se ejecutan en el contexto de la página visitada— y un script de fondo, que coordina la lógica de la extensión y sus comunicaciones con servicios externos. Ese modelo de permisos delimita tanto lo que la extensión puede automatizar dentro del navegador como lo que queda fuera de su alcance sin intervención directa del usuario o del sistema operativo.

2.3. RECONOCIMIENTO Y SÍNTESIS DE VOZ (STT Y TTS)

El reconocimiento automático del habla (automatic speech recognition, ASR; también conocido como speech to text, STT) es el proceso mediante el cual un sistema decodifica una señal de audio con voz humana y la transcribe a texto, típicamente combinando un modelo acústico —que traduce la señal a una secuencia probable de fonemas— y un modelo de lenguaje —que determina, a partir de esos fonemas, las palabras más probables— [IBM, 2024b].

La síntesis de voz (text to speech, TTS) es el proceso inverso: toma una entrada textual y la convierte en audio hablado. Esa entrada no siempre es texto plano: estándares como SSML (Speech Synthesis Markup Language), del W3C, permiten anotar el texto con marca-

dores de énfasis, acento, velocidad o pausas, para un control más fino de cómo se pronuncia [Burnett y Shuang, 2010]. A nivel de técnica, la síntesis de voz se ha resuelto históricamente de más de una forma: por concatenación de fragmentos de voz pregrabada, por síntesis paramétrica a partir de un modelo estadístico de la voz, y, más recientemente, mediante modelos neuronales generativos que producen la forma de onda directamente a partir de datos de entrenamiento [Tan *et al.*, 2021].

Lectores de pantalla como JAWS o NVDA no necesariamente implementan su propio motor de síntesis: en general, delegan la conversión final de texto a voz a un motor TTS del sistema operativo o a una voz de terceros instalada por el usuario, y lo que le corresponde al lector de pantalla es determinar qué texto de la interfaz enviar a ese motor.

2.4. IA MULTIMODAL: MODELOS DE LENGUAJE VISUAL (VLM)

2.4.1. IA generativa

La IA generativa es un tipo de inteligencia artificial capaz de producir contenido nuevo — texto, imágenes u otro tipo de dato— a partir de patrones aprendidos de grandes volúmenes de datos, **en lugar de limitarse a clasificar o predecir una etiqueta sobre datos ya existentes** [IBM, 2024a].

En el caso particular de los modelos de lenguaje, esa generación se implementa técnicamente como una predicción. Un modelo autoregresivo —que genera cada nueva unidad de texto a partir de lo que él mismo generó **pre**viamente—, como los que se basan en la arquitectura Transformer —una arquitectura de red neuronal que pondera mediante **mecanismos de atención** qué partes de la entrada son más relevantes para cada predicción—, no produce una respuesta completa de una sola vez: predice, token a token —el token es la unidad mínima de texto en que el modelo divide el lenguaje, por ejemplo una palabra o parte de ella—, cuál es la unidad de texto más probable dado todo lo que ha recibido y generado hasta ese momento, y repite ese proceso hasta completar la respuesta [Vaswani *et al.*, 2017]. Es decir, “generar” contenido y “predecir la palabra siguiente” no son dos capacidades distintas, sino una relación de mecanismo a resultado: el modelo predice de forma iterativa, token a token, y esa cadena de predicciones es percibida por quien lo usa como la generación de una respuesta nueva.

2.4.2. IA multimodal

La IA multimodal corresponde a **modelos** capaces de procesar y/o producir información de más de un tipo de dato —texto, imágenes, audio o video— de forma conjunta, en lugar de limitarse a un único tipo de entrada o salida [Wu *et al.*, 2023]. Dentro de esa familia se encuentran los modelos de lenguaje visual (VLM, vision language model), especializados en la

interacción entre texto e imágenes: reciben como entrada una combinación de imagen(es) y texto, y devuelven una salida en texto, apoyándose en una representación interna que alinea los conceptos visuales de la imagen con los conceptos del lenguaje [Wu *et al.*, 2023].

2.4.3. Inferencia

La inferencia es el acto de usar un modelo de IA **ya entrenado** para obtener una salida a partir de una entrada nueva [NVIDIA, 2024]. Ejecutar esa inferencia sobre un modelo de gran tamaño es, en sí mismo, un desafío computacional: un equipo de escritorio promedio no puede hacerlo, o al menos no en tiempos **aceptables**, por lo que habitualmente se recurre a hardware específico —típicamente GPU— para hacerla eficiente. Frente a esto existen **dos** caminos habituales, **ambos con inconvenientes**: los servicios de IA en línea **corren** sobre datacenters de gran escala y cobran por consulta; además, procesan los datos del usuario en infraestructura de terceros, sin garantías claras sobre su **tratamiento**.

En este contexto aparece el concepto de latencia: el tiempo de espera entre que se envía una consulta y se recibe la respuesta. **La latencia es un factor crítico para la experiencia de uso y se retoma más adelante.**

2.5. EVALUACIÓN AUTOMATIZADA DE VLMs

Existe una gran oferta de modelos VLM **open source**, y distintos modelos entregan resultados diferentes para las mismas consultas, por lo que elegir el más **pertinente** para una necesidad **determinada** no es trivial. **Hacerlo a mano** —tomar un VLM, **hacerle** N preguntas y evaluar cada una de esas N respuestas— no escala: para que el resultado sea confiable se necesita un N grande, y probar varios modelos vuelve la evaluación manual una tarea repetitiva e inviable. De ahí la necesidad, documentada en la literatura, de automatizar este proceso mediante técnicas de benchmarking sistemático.

2.5.1. Rúbrica cualitativa

Una rúbrica es un instrumento de evaluación cualitativa que describe, para cada nivel de desempeño posible, los criterios concretos que una respuesta debe cumplir para alcanzarlo; **se construye a mano** antes de la evaluación y permite calificar de forma consistente incluso cuando la respuesta es abierta y no tiene una única forma correcta **—por ejemplo, al calificar la respuesta de un estudiante en un examen de argumentación, donde una rúbrica puede distinguir niveles que van desde una postura sin fundamento hasta un argumento completo y bien sustentado.**

2.5.2. LLM-as-judge

“LLM-as-a-judge” es un paradigma de evaluación en el que un LLM, en lugar de un evaluador humano, cumple el rol de juez: se le entrega la respuesta que se quiere evaluar junto con los criterios de evaluación —por ejemplo, una rúbrica—, y el modelo determina si esos criterios están presentes o ausentes para asignar un puntaje de forma cuantitativa [Zheng *et al.*, 2023].

2.5.3. Judge evaluation

Para confirmar que un LLM evaluador funciona bien, en este trabajo se introduce el concepto de “Judge evaluation”: un humano califica de forma independiente las mismas respuestas que evalúa el LLM, y luego se contrasta cuánto se acercan los puntajes de uno y otro. Es una segunda capa de evaluación, destinada a comprobar que el criterio con el que el LLM juzga esas respuestas está bien calibrado.

Con la judge evaluation se cierra el ciclo: el humano evalúa las respuestas, el LLM las evalúa también, y la comparación entre ambos permite validar al LLM evaluador a partir del criterio humano.

Respecto de los sistemas estandarizados de evaluación específica para VLMs, ya existen herramientas y métricas destinadas a esta tarea. Un ejemplo es VLMEvalKit (open-compass): un toolkit open source de evaluación de modelos multimodales que unifica la evaluación de una gran cantidad de modelos sobre decenas de benchmarks con configuración mínima, automatizando la preparación de datos, la inferencia y el cálculo de métricas, e incluso usando un LLM como extractor de respuestas cuando la comparación exacta falla [Duan *et al.*, 2024]. Sin embargo, ninguna de estas herramientas está diseñada específicamente para la lectura de sitios web **en el contexto de la discapacidad visual**, que es el foco de este trabajo.

CAPÍTULO 3
PROPUESTA DE SOLUCIÓN

Se debe desarrollar la solución propuesta. Los subcapítulos por poner aquí son propios del autor. Se sugiere mencionar metodología usada. Es conveniente incorporar figuras y tablas para aclarar la solución, que deben indicar el número de la figura, su nombre y su autor o fuente (si las diseñas tú, la fuente es “Elaboración propia”). Ver ejemplos en esta página y en la siguiente.

Cabe mencionar que aquí está la esencia del trabajo en lo que se refiere al aporte creativo del memorista, es el momento de demostrar que usted es un destacado profesional que creó, diseñó y/o llevó a cabo la solución propuesta.

EJEMPLO DE COMO CITAR FIGURAS E ILUSTRACIONES

Se colocó una imagen que se puede referenciar también desde el texto (Ver figura 1).

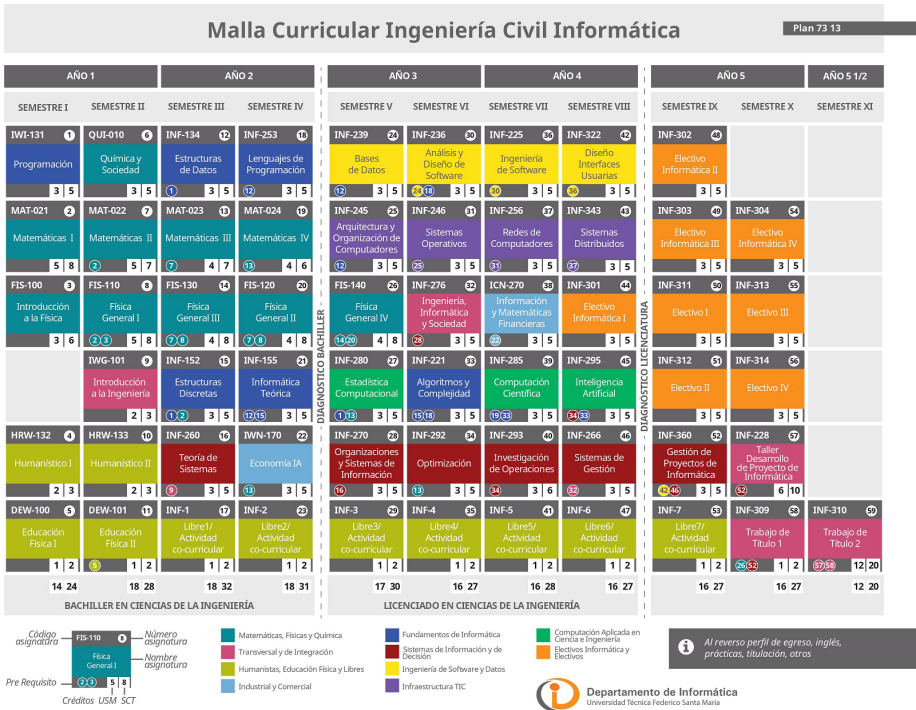


Figura 1: Malla Curricular Ingeniería Civil Informática.
Fuente: Departamento de Informática.

En base a los antecedentes presentados en los capítulos anteriores, se plantea desarrollar un programa que, al presionar un atajo de teclado, le pregunte al usuario qué necesita saber de la pantalla, y que el computador le responda. Ese comando lo recibe una extensión de navegador, que envía una captura de la pantalla junto con la pregunta a un servidor de inferencia; este la procesa y retorna una respuesta en texto que, finalmente, es transformada a audio por el lector de pantalla del propio usuario. Las secciones siguientes describen, en ese orden, la arquitectura general del sistema, la extensión de navegador, el backend que procesa las consultas, la infraestructura sobre la que corre el modelo, y el proceso de selección del modelo VLM utilizado.

3.1. ARQUITECTURA GENERAL DEL RELATOR DE PANTALLA

La arquitectura propuesta para el sistema se compone de una extensión de navegador y un backend que coordinan la captura, el envío al modelo y la entrega de la respuesta.

La extensión de navegador se encarga de capturar la pantalla ante el input del usuario. Esa captura pasa al backend, que toma el identificador del usuario (o de la pestaña de navegador activa), la captura y la pregunta, y las envía al servidor de inferencia con GPU.

Una vez procesada, la información vuelve al backend, que identifica a quién pertenece según el ID y devuelve el texto de respuesta al usuario correspondiente. Finalmente, ese texto es transformado a audio por el lector de pantalla del usuario. Cabe precisar que el paso de texto a voz (TTS) queda fuera del flujo de la arquitectura, ya que lo realiza el propio lector de pantalla.

En resumen, el flujo es: sitio -> extensión (captura + entrada/salida de voz) -> backend (ID + captura + pregunta) -> servidor de inferencia con GPU (VLM) -> backend (enruta según ID) -> texto de respuesta -> lector de pantalla (TTS, externo al sistema).

3.2. EXTENSIÓN DE NAVEGADOR

Se desarrolló una extensión diseñada para navegadores basados en Chromium, que captura el viewport visible o la página completa mediante atajos de teclado configurables por el usuario.

Para la captura de página completa se aplica una lógica de scroll & stitch: se congelan los elementos dinámicos del sitio y se aplican desplazamientos (scrolls) que permiten recorrer secuencialmente el estado actual de la página, tomando una captura en cada instante y uniéndolas luego en una sola imagen. Esto entrega la perspectiva completa y actual del sitio, según lo que requiera la consulta del usuario.

El output de las imágenes puede entregarse directamente en un formato de imagen (por

Arquitectura general del relator de pantalla

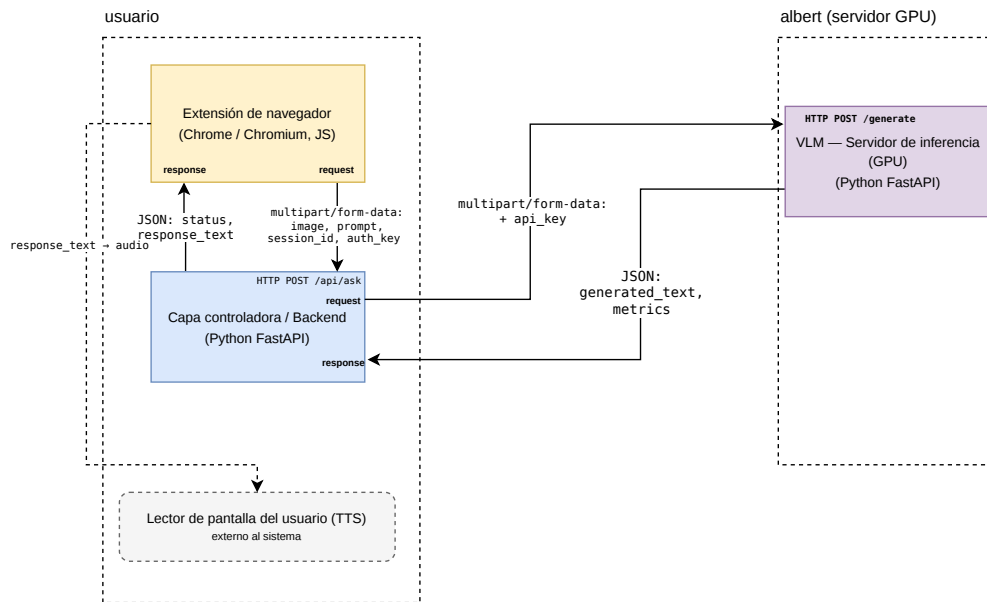


Figura 2: Arquitectura general del relator de pantalla.

Fuente: Elaboración propia.

ejemplo .jpg o .png) o convertirse a base64 para enviarlo más fácilmente por HTTP al backend.

figura: esquema del scroll & stitch (congelado de elementos dinámicos, recorrido por scroll, capturas parciales y ensamblado en una imagen). Fuente: Elaboración propia.

Además de capturar la pantalla, la extensión es responsable de la modalidad de interacción con el usuario, es decir, de si la consulta se hace por texto o por voz. Esta responsabilidad es exclusiva de la extensión: es irrelevante para el backend si la pregunta llegó escrita o hablada, dado que en ambos casos recibe únicamente texto. El desarrollo de esta capa de interacción se planteó en tres iteraciones, cada una funcional por sí sola:

3.2.1. textToText (TTT)

Se envía texto plano, escrito manualmente en un cuadro de input mostrado por la extensión, al backend, y se recibe la respuesta también en texto, mostrada en un pop-up para la lectura del usuario. Esta iteración es válida solo para pruebas de desarrollador (con visión completa).

Flujo de la información del relator de pantalla

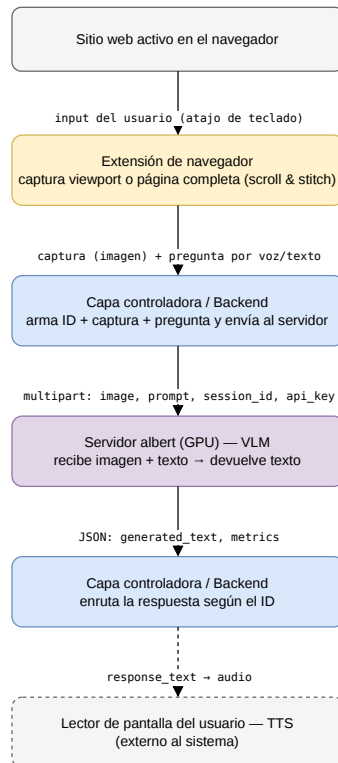


Figura 3: Flujo de la información del relator de pantalla.
Fuente: Elaboración propia.

3.2.2. textToVoice (TTV)

Se envía texto plano escrito manualmente y se recibe audio de vuelta, en forma de TTS procesado por el lector de pantalla. Es la primera iteración válida para pruebas con usuarios reales.

3.2.3. voiceToVoice (VTV)

Iteración definitiva. Integra la entrada por voz vía STT —usando la Web Speech API de Google, que procesa la voz del usuario localmente o en la nube— y la salida de audio procesada por el TTS del lector de pantalla.

Las iteraciones TTV y VTV pueden convivir en el mismo software empaquetado para el usuario: queda a su criterio si prefiere escribir las preguntas o hacerlas por voz. Esta flexibilidad importa porque dejar de escribir para hablarle al computador a mitad de una sesión puede romper el flujo habitual de trabajo del usuario.

figura/tabla: comparación de las tres iteraciones (input, output, público de prueba) o capturas de la interfaz (cuadro de input y pop-up de respuesta). Fuente: Elaboración propia.

3.3. BACKEND DEL SISTEMA

El backend es el servidor que recibe las consultas provenientes de la extensión y coordina su procesamiento: recibe siempre una imagen (la captura) junto con texto (la pregunta), y devuelve texto (la respuesta). Es, en ese sentido, la capa de procesamiento de IA del sistema: mientras la extensión resuelve todo lo relativo a la interacción con el usuario (front), el backend se limita a procesar la consulta y devolver una respuesta (back), sin distinguir si la modalidad de entrada del usuario fue texto o voz.

Su naturaleza es la de una implementación funcional para validar el pipeline completo, no el producto final.

3.4. INFRAESTRUCTURA Y SERVIDOR ALBERT

La infraestructura de ejecución del proyecto contempla dos partes: los equipos de los usuarios y el servidor de inferencia con GPU.

Del lado del usuario, basta un equipo con un navegador basado en Chromium; el sistema operativo puede ser Windows, macOS, Linux o cualquiera compatible con dichos navegadores.

Para la inferencia se utilizó “albert”, una máquina del laboratorio de data science (por confirmar) que cuenta con una tarjeta gráfica NVIDIA A40 con 48 GB de VRAM, capaz de correr modelos VLM de alto nivel. Se accede a albert vía jump host desde los equipos de desarrollo para enviar las consultas mediante conexión SSH, proceso que puede automatizarse a través de un túnel SSH.

3.4.1. Decisión de infraestructura

Al determinar la arquitectura de inferencia existían tres formas de resolverla: usar una API o una IA ya empaquetada de un tercero, levantar un servidor propio remoto (en la nube), o utilizar un servidor propio local. Se optó por un servidor propio, ya que permite controlar tanto el modelo utilizado como la infraestructura sobre la que corre, sin depender de las condiciones de un tercero ni de los costos por consulta de un servicio en línea.

Antes de llegar a un servidor local dedicado se probaron otras dos alternativas para ejecutar un modelo VLM acotado. Primero, el equipo de desarrollo local, usando la CPU (Ryzen 5

2600), ya que su GPU no cuenta con CUDA cores compatibles con los modelos evaluados: los tiempos de respuesta no fueron aceptables. Se probó luego con un equipo más potente pero sin GPU dedicada, con resultados igual de insuficientes, lo que llevó a concluir que una GPU era obligatoria para este trabajo. Como alternativa a comprar o mantener hardware propio, se evaluaron los servidores de Google Colab, con el entorno de GPU NVIDIA T4 (16 GB de VRAM); si bien el desempeño mejoró notoriamente, escalar esa opción a un uso sostenido implica costos y una gestión de infraestructura poco prácticos para este trabajo. Finalmente se optó por “albert”, una máquina del laboratorio de data science (por confirmar) que cuenta con una tarjeta gráfica NVIDIA A40 con 48 GB de VRAM, capaz de correr modelos VLM de alto nivel, y a la que se accede vía jump host desde los equipos de desarrollo mediante conexión SSH, proceso que puede automatizarse a través de un túnel SSH.

Para una misma prueba, los tiempos fueron: aproximadamente 30 minutos en la CPU Ryzen, 1 minuto en la T4 de Google Colab y tan solo 15 segundos en la A40 de albert.

Esos 15 segundos coinciden con el límite de tiempo deseado para la interacción con humanos: por encima de ese umbral se tiende a perder la atención del usuario. Este es el concepto de latencia (tiempo de espera por consulta) introducido en el Capítulo 2, ahora como requisito operativo del sistema.

fuentes pendientes: respaldar el umbral de 15 s de atención/latencia con una referencia; confirmar especificaciones exactas de albert y del equipo de desarrollo.

3.5. BENCHMARK DE MODELOS VLM

Se realizó un procedimiento de benchmarking para seleccionar el VLM más adecuado para la tarea de analizar sitios web con perspectiva de accesibilidad, evaluando sobre sitios reales.

3.5.1. Corpus y preguntas

Se armó un corpus de 10 sitios web en 4 categorías:

- Instituciones educativas: USM, UCH.
- Organizaciones estatales: SII, SENADIS.
- Sitios de e-commerce: Falabella, Mercado Libre, Ticketmaster, Ticketplus.
- Foros y comunidades: Reddit, 9gag.

Para cada categoría se escribieron 3 preguntas generales, y para cada sitio 3 preguntas específicas, sumando 6 preguntas por sitio. En total: 6 preguntas × 10 sitios = 60 preguntas por cada modelo evaluado.

3.5.2. Rúbrica de evaluación

Se escribieron rúbricas con puntajes del 0 al 3 según **qué tanto se acerca la respuesta a la de un ser humano con visión completa: 0 alucinación o falsedad, 1 parcial, 2 correcta pero incompleta, 3 exacta.** La rúbrica se rige por los datos literales (se debe extraer el contenido exacto de los textos; si se cambia una palabra, aunque signifique lo mismo, se considera incorrecto) y por la no verbosidad (no se mejora el puntaje de una respuesta solo por ser extensa).

3.5.3. Modelos evaluados

Se parte probando con modelos pequeños, como moondream2 e InternVL2-2B, que suelen alucinar información y rara vez entregan respuestas de nivel 3. Luego se migra a modelos con más parámetros y mayor uso de GPU, como InternVL3-2B y Qwen3-VL-8B.

3.5.4. Pipeline y selección del juez (LLM-as-a-judge)

El pipeline de evaluación sigue el flujo: **screenshots ->VLM ->JSON ->juez LLM ->métricas.**

Para la capa de evaluación por juez se probaron distintos modelos LLM contra un set de respuestas previamente evaluadas por un humano. **Se eligió como juez a Gemma3-12B (por confirmar).**

Para elegir el juez se usaron métricas como el porcentaje de exact match (que el LLM asigne exactamente el mismo puntaje que el humano), que alcanzó un máximo de 72,5 % con Gemma3. También se aplicaron métricas de error tipo MAE y RMSE.

Se realizó un proceso de mejora continua de prompts para aumentar el exact match y reducir los errores del juez, iterando 20 veces hasta alcanzar el **tope técnico** de Gemma3. Los criterios considerados para el **desarrollo del prompt** de evaluación fueron: precisión espacial, razonamiento semántico y resistencia a alucinaciones.

3.5.5. Resultado de la selección

Elegido Gemma3 como juez, se corrió nuevamente el benchmark con él sobre el universo completo de modelos candidatos. Dado que no se logró un exact match de 80 % o superior, se concluye que el instrumento de benchmark automatizado no es lo bastante confiable para elegir de forma autónoma UN único mejor modelo. Sin embargo, sí resulta útil para lo que en la práctica es su aporte principal: acotar el universo de modelos posibles —a modo ilustrativo, de un orden de 1.000 modelos VLM disponibles a un conjunto acotado de unos 100— antes de pasar a un análisis manual. Ese filtrado reduce drásticamente el trabajo de revisión manual, que de otra forma sería inviable sobre el universo completo; lo que se hace a mano sobre ese conjunto acotado es, en esencia, lo mismo que hace el juez LLM de forma automatizada, con la diferencia de que en ese instrumento —el criterio humano— sí se puede confiar. Sobre el conjunto de 10 modelos preseleccionados de esta forma se eligió finalmente Qwen3-VL-8B.

CAPÍTULO 4

VALIDACIÓN DE LA SOLUCIÓN

Se debe validar la solución propuesta. Esto significa probar o demostrar que la solución propuesta es válida para el entorno donde fue planteada.

Tradicionalmente es una etapa crítica, pues debe comprobarse por algún medio que vuestra propuesta es básicamente válida. En el caso de un desarrollo de software es la construcción y sus pruebas; en el caso de propuestas de modelos, guías o metodologías podrían ser desde la aplicación a un caso real hasta encuestas o entrevistas con especialistas; en el caso de mejoras de procesos u optimizaciones, podría ser comparar la situación actual (previa a la memoria) con la situación final (cuando la memoria está ya implementada) en base a un conjunto cuantitativo de indicadores o criterios.

4.1. EJEMPLO DE COMO CITAR TABLAS

Se colocó una tabla que se puede referenciar también desde el texto (Ver tabla 1).

Tabla 1: Coloquios del Ciclo de Charlas Informática.
Fuente: Elaboración Propia.

Título Coloquio	Presentador, País
"Sensible, invisible, sometimes tolerant, heterogeneous, decentralized and interoperable... and we still need to assure its quality..."	Guilherme Horta Travassos, Brasil.
"Dispersed Multiphase Flow Modeling: From Environmental to Industrial Applications"	Orlando Ayala, EE.UU.
"Líneas de Producto Software Dinámicas para Sistemas atentos el Contexto"	Rafael Capilla, España.
...	...

4.2. ENFOQUE DE LA VALIDACIÓN

La validación de este trabajo combina dos ejes complementarios. El primero consiste en un conjunto de métricas técnicas, calculadas directamente por el autor sobre el sistema construido: la exactitud del reconocimiento de voz (STT), la latencia del backend, la latencia de

captura de pantalla y la pertinencia de las respuestas entregadas por el modelo. El segundo consiste en pruebas con usuarios reales con discapacidad visual, relatadas como entrevistas: se presenta el sistema al usuario, este intenta un conjunto de tareas reales de interpretación de imágenes en sitios web integradas a su flujo de trabajo habitual con su lector de pantalla, y valora cada interacción.

Dado que el número de usuarios disponibles para las pruebas es reducido (3 personas) y su selección no siguió una metodología de muestreo estadístico, las entrevistas no se presentan como un procedimiento estadístico ni sus resultados se usan para declarar la solución “válida” o “no válida” **en términos porcentuales**. Su objetivo es más acotado: mostrar, mediante casos concretos, si el sistema funciona o no en la práctica y de qué forma, complementando así el eje de métricas técnicas con la **perspectiva de uso real**.

4.3. MÉTRICAS TÉCNICAS

Estas métricas se calculan directamente por el autor sobre el funcionamiento del sistema, sin depender de la evaluación de terceros ni de la opinión de los usuarios.

4.3.1. Exactitud del reconocimiento de voz (STT)

Se mide sobre un conjunto de N palabras habladas por el usuario durante las pruebas en modalidad de voz (VTV, ver Capítulo 3), contabilizando cuántas de ellas fueron transcritas incorrectamente por el motor de STT utilizado por la extensión.

pendiente: definir N, ejecutar la medición y reportar la tasa de error resultante (palabras erróneas / N).

4.3.2. Latencia del backend

Se mide el tiempo transcurrido entre que la extensión envía una consulta (captura de pantalla y pregunta) al backend, y el momento en que este devuelve la respuesta en texto.

pendiente: ejecutar la medición sobre un conjunto de consultas y reportar el tiempo promedio y su dispersión.

4.3.3. Latencia de captura

Se compara el tiempo que toma la extensión en capturar el viewport visible de la pantalla **frente al tiempo** que toma capturar la página completa mediante el mecanismo de scroll &

stitch (ver Capítulo 3).

pendiente: ejecutar la medición y reportar ambos tiempos.

4.3.4. Pertinencia de las respuestas

Se evalúa qué tan pertinente resulta la respuesta entregada por el sistema respecto de lo que el usuario efectivamente preguntó, reutilizando el criterio de la rúbrica cualitativa definida en el Capítulo 3 (ver 3.5), aplicado esta vez sobre las consultas reales realizadas durante las pruebas con usuarios.

pendiente: ejecutar la medición sobre las consultas de las sesiones y reportar los resultados.

4.4. PRUEBAS CON USUARIOS: ENTREVISTAS

4.4.1. Diseño

Se estableció un número objetivo de entre 3 y 5 usuarios. Se **exigió** que todos presentaran discapacidad visual severa o ceguera total, que fueran usuarios habituales de lectores de pantalla y que tuvieran un nivel de manejo computacional intermedio o avanzado.

Como entorno de pruebas se utilizó un subconjunto del corpus del benchmark (ver 3.5): al menos 2 sitios de categorías distintas y al menos 2 tareas por cada sitio. Además, **se exigió utilizar al menos una vez la modalidad por voz (VTV)**.

El detalle de las tareas por sesión se presenta en una tabla en el anexo. **REFERENCIA PENDIENTE: insertar ?? a la tabla del anexo correspondiente.**

4.4.2. Instrumento: puntaje de satisfacción

Al terminar cada tarea se le pregunta al usuario su nivel de satisfacción con la respuesta obtenida, en un número entero entre **1 y 5**, denominado puntaje de satisfacción (PS): 1 corresponde a insatisfacción total (el relator de pantalla **alucinó información** o su respuesta fue de nula ayuda para completar la tarea), 3 a satisfacción parcial (el relator de pantalla cumplió con lo mínimo solicitado, pero hay espacio de mejora) y 5 a satisfacción total (la respuesta fue la esperada y ayudó a completar la tarea sin problemas). Este puntaje se reporta como parte del relato de cada entrevista, sin agregarlo en un promedio o porcentaje global.

4.4.3. Participantes

La validación se realizó finalmente con 3 participantes, dentro del rango objetivo definido. Los tres presentan ceguera total y usan JAWS como lector de pantalla. Se diferencian entre sí en las condiciones asociadas a su discapacidad visual y en su nivel de manejo computacional (uno con nivel medio y dos con nivel avanzado).

4.4.4. Sesiones

Sobre estos 3 participantes se realizaron 4 sesiones en total, ya que P2 realizó dos sesiones. De ellas, 3 se hicieron en modalidad texto (TTV) y 1 en modalidad voz (VTV).

4.4.5. Caso 1: primer usuario (P1)

La primera sesión se realizó con P1 (nivel de manejo computacional medio), en modalidad texto (TTV).

PENDIENTE: narrar en formato de entrevista la sesión de P1 – tareas concretas intentadas, cómo respondió el relator de pantalla en cada una, y el puntaje de satisfacción (1-5) que P1 asignó a cada tarea. Al ser la primera sesión ejecutada, sirve como caso ilustrativo de referencia sobre la impresión inicial del sistema frente a un usuario real.

4.4.6. Casos 2 a 4: P2 (dos sesiones) y P3

PENDIENTE: narrar en el mismo formato de entrevista (tareas intentadas, respuesta del sistema, puntaje de satisfacción por tarea) las sesiones de P2 – su primera sesión en modalidad texto (TTV) y su segunda, la única de las 4 realizada en modalidad voz (VTV) – y de P3.

A modo de resumen de las 19 tareas intentadas a lo largo de las 4 sesiones, las puntuaciones de satisfacción se concentraron mayoritariamente en el rango medio-alto (3 a 5), aunque también hubo tareas con puntajes bajos, coherentes con los modos de falla identificados a continuación.

4.5. ANÁLISIS DE PATRONES

A partir de la revisión de los registros de las sesiones se identificaron patrones en el desempeño del sistema. Por categoría de sitio, el mejor desempeño se observó en sitios de

e-commerce y redes sociales, mientras que el peor desempeño se dio en sitios educativos, particularmente en portales de gestión interna que requieren autenticación. Por tipo de tarea, el sistema tuvo buen desempeño en tareas descriptivas o de extracción de información –por ejemplo, “qué es”, “qué hay” o “describe”–, y un desempeño deficiente en tareas procedimentales o de navegación –por ejemplo, “cómo hago”, “dónde voy” o solicitudes de pasos a seguir–.

Entre los modos de falla identificados se cuentan: un sesgo de perspectiva vidente en las respuestas del modelo, alucinaciones cuando el elemento consultado no está visible en la captura, descripciones superficiales, y fallas del mecanismo de scroll & stitch (ver Capítulo 3, sección de extensión de navegador) al capturar ventanas emergentes (no pop-ups, sino ventanas independientes usualmente para una sola función, como por ejemplo pagos online o visualizar documentos).

En la modalidad de voz se observó un comportamiento particular: se reportó una satisfacción más alta en la misma tarea ejecutada por texto incluso cuando esta no se completaba. Esto sugiere que la experiencia de interacción por voz tiene un valor intrínseco, independiente del éxito de la tarea.

4.6. RETROALIMENTACIÓN DE LOS USUARIOS

Los participantes aportaron las siguientes sugerencias de mejora durante las sesiones:

- Contar con una versión móvil y compatible con aplicaciones, no solo con navegadores de escritorio.
- Que el chat mantuviera memoria de la conversación, para poder seguir preguntando sobre un mismo elemento sin repetir contexto.
- Poder acceder o navegar directamente a un enlace ya identificado por el sistema.
- Mejorar el feedback del sistema de entrada por voz, para que el usuario sepa si lo que dice se está capturando correctamente.

4.7. CONCLUSIÓN DE LA VALIDACIÓN

Esta validación no busca declarar la solución “aprobada” en base a un umbral estadístico, sino reunir evidencia, desde dos ángulos complementarios, de si el relator de pantalla funciona en la práctica. **PENDIENTE: una vez completadas las métricas técnicas (exactitud de STT, latencia del backend, latencia de captura y pertinencia de las respuestas), sintetizar aquí sus resultados.** Por el lado de las entrevistas, los casos relatados muestran un sistema

que responde razonablemente bien en tareas descriptivas y de extracción de información, particularmente en sitios de e-commerce y redes sociales, y que tiene un desempeño más débil en tareas procedimentales y en sitios que requieren autenticación.

Los modos de falla identificados en la sección de análisis de patrones y las sugerencias reportadas por los propios participantes se abordan como trabajo futuro.

CAPÍTULO 5

CONCLUSIONES

Las Conclusiones son, según algunos especialistas, el aspecto principal de una memoria, ya que reflejan el aprendizaje final del autor del documento. En ellas se tiende a considerar los alcances y limitaciones de la propuesta de solución, establecer de forma simple y directa los resultados, discutir respecto a la validez de los objetivos formulados, identificar las principales contribuciones y aplicaciones del trabajo realizado, así como su impacto o aporte a la organización o a los actores involucrados. Otro aspecto que tiende a incluirse son recomendaciones para quienes se sientan motivados por el tema y deseen profundizarlo, o lineamientos de una futura ampliación del trabajo.

Todo esto debe sintetizarse en al menos 5 páginas.

ANEXOS

En los Anexos se incluye todo aquel material complementario que no es parte del contenido de los capítulos de la memoria, pero que permiten a un lector contar con un contenido adjunto relacionado con el tema.

5.1. ENCUESTA SOBRE TECNOLOGÍAS PARA PERSONAS CIEGAS

Este anexo detalla el instrumento mencionado en el Capítulo 1 (Definición del problema), aplicado para validar empíricamente la problemática social antes de proponer una solución.

La encuesta se estructuró en 4 secciones:

- **PENDIENTE:** nombre y contenido de la Sección 1 (perfil general del encuestado: condición, nivel de discapacidad visual, lector de pantalla utilizado, manejo computacional autoreportado).
- **PENDIENTE:** nombre y contenido de la Sección 2 (hábitos de uso y dificultades reportadas al navegar la web con lector de pantalla).
- **PENDIENTE:** nombre y contenido de la Sección 3 (preguntas generales sobre necesidades y funcionalidades de accesibilidad más valoradas).
- **PENDIENTE:** nombre y contenido de la Sección 4 (preguntas específicas por tipo de sitio web).

PENDIENTE: detallar la diferencia entre las preguntas de tipo general (aplicadas a todos los encuestados) y las preguntas específicas (según el tipo de sitio web consultado), con ejemplos concretos de cada una.

REFERENCIAS BIBLIOGRÁFICAS

- [Abou-Zahra y World Wide Web Consortium (W3C) Web Accessibility Initiative, 2024] Abou-Zahra, S. y World Wide Web Consortium (W3C) Web Accessibility Initiative (2024). Tools and techniques. <https://www.w3.org/WAI/people-use-web/tools-techniques/>. Actualizada el 25 de junio de 2024.
- [Beaumont, 1990] Beaumont, R. H. (1990). Patient preference for waxed or unwaxed dental floss. *Journal of periodontology*, 61(2):123–125.
- [Bourne et al., 2021] Bourne, R. R. A., Steinmetz, J. D., Flaxman, S., y GBD 2019 Blindness and Vision Impairment Collaborators (2021). Trends in prevalence of blindness and distance and near vision impairment over 30 years: an analysis for the global burden of disease study. *The Lancet Global Health*, 9(2):e130–e143.
- [Burnett y Shuang, 2010] Burnett, D. C. y Shuang, Z. W. (2010). Speech synthesis markup language (ssml) version 1.1. Technical report, World Wide Web Consortium. W3C Recommendation, 7 de septiembre de 2010.
- [Campbell et al., 2024] Campbell, A., Adams, C., Montgomery, R. B., Cooper, M., y Kirkpatrick, A. (2024). Web content accessibility guidelines (wcag) 2.2. Technical report, World Wide Web Consortium. W3C Recommendation, 12 de diciembre de 2024.
- [Chromium Blog, 2010] Chromium Blog (2010). A year of extensions. <https://blog.chromium.org/2010/12/year-of-extensions.html>. 9 de diciembre de 2010.
- [Duan et al., 2024] Duan, H., Fang, X., Yang, J., Zhao, X., Ma, Z., Qiao, Y., Li, M., Liang, T., Zhu, L., Chen, Z., y others (2024). VLMEvalKit: An open-source toolkit for evaluating large multi-modality models. *arXiv preprint arXiv:2407.11691*.
- [Freedom Scientific, 2024] Freedom Scientific (2024). Jaws – screen reading software. <https://www.freedomscientific.com/products/software/jaws/>.
- [Georget et al., 1994] Georget, D., Parker, R., y Smith, A. (1994). A study of the effects of water content on the compaction behaviour of breakfast cereal flakes. *Powder Technology*, 81(2):189–195.
- [Gettelfinger y Cussler, 2004] Gettelfinger, B. y Cussler, E. (2004). Will humans swim faster or slower in syrup? *AIChE journal*, 50(11):2646–2647.
- [IBM, 2024a] IBM (2024a). Generative ai vs. predictive ai: What's the difference? <https://www.ibm.com/think/topics/generative-ai-vs-predictive-ai-whats-the-difference>.
- [IBM, 2024b] IBM (2024b). What is speech recognition? <https://www.ibm.com/think/topics/speech-recognition>.

- [Mace y The Center for Universal Design, 1997] Mace, R. y The Center for Universal Design (1997). The 7 principles of universal design. Technical report, North Carolina State University. Mirror publicado por el Centre for Excellence in Universal Design (CEUD), National Disability Authority, Irlanda.
- [MDN Web Docs, 2024a] MDN Web Docs (2024a). Accessibility tree. https://developer.mozilla.org/en-US/docs/Glossary/Accessibility_tree.
- [MDN Web Docs, 2024b] MDN Web Docs (2024b). Aria. <https://developer.mozilla.org/en-US/docs/Web/Accessibility/ARIA>.
- [MDN Web Docs, 2024c] MDN Web Docs (2024c). Webextensions. <https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions>.
- [Nurthen et al., 2023] Nurthen, J., Cooper, M., y Henry, S. L. (2023). Wai-aria 1.2. Technical report, World Wide Web Consortium (W3C). W3C Recommendation, 6 de junio de 2023.
- [NVIDIA, 2024] NVIDIA (2024). What is ai inference? <https://www.nvidia.com/en-us/glossary/ai-inference/>.
- [Schwab, 2002] Schwab, I. R. (2002). Cure for a headache. *British Journal of Ophthalmology*, 86(8):843-843.
- [Tan et al., 2021] Tan, X., Qin, T., Soong, F., y Liu, T.-Y. (2021). A survey on neural speech synthesis. *arXiv preprint arXiv:2106.15561*.
- [Vaswani et al., 2017] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., y Polosukhin, I. (2017). Attention is all you need. En *Advances in Neural Information Processing Systems*, volumen 30.
- [WebAIM, 2024] WebAIM (2024). Screen reader user survey #10 results. <https://webaim.org/projects/screenreadersurvey10/>.
- [WHATWG, 2026] WHATWG (2026). Dom standard. <https://dom.spec.whatwg.org/>. Living Standard.
- [World Health Organization, 2026] World Health Organization (2026). Blindness and vision impairment. <https://www.who.int/news-room/fact-sheets/detail/blindness-and-visual-impairment>. Fact sheet, actualizada el 10 de febrero de 2026.
- [World Wide Web Consortium (W3C), 2021] World Wide Web Consortium (W3C) (2021). Forming the webextensions community group. <https://www.w3.org/community/webextensions/2021/06/04/forming-the-wecg/>. Anuncio de formación del grupo comunitario, 4 de junio de 2021.
- [World Wide Web Consortium (W3C), Web Accessibility Initiative, 2014] World Wide Web Consortium (W3C), Web Accessibility Initiative (2014). Resources on alternative text for images. <https://www.w3.org/WAI/alt/>.

- [Wu *et al.*, 2023] Wu, J., Gan, W., Chen, Z., Wan, S., y Yu, P. S. (2023). Multimodal large language models: A survey. *arXiv preprint arXiv:2311.13165*.
- [Zheng *et al.*, 2023] Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E., Zhang, H., Gonzalez, J. E., y Stoica, I. (2023). Judging LLM-as-a-judge with MT-bench and chatbot arena. En *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, volumen 36.